

Reviewer Pack — Minimal Rank of Geometric Quantization over Symplectic Manifolds

Entry 93f863499760 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-22 22:16:24 UTC

Minimal Rank of Geometric Quantization over Symplectic Manifolds vs. Sum-of-Squares Hierarchy for Max-CUT

Entry ID: 93f863499760 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-22 22:16:24 UTC

1. Conjecture

Field A (mathematical branch): Geometric Quantization Theory **Field B** (complexity object): Complexity Theory: Sum-of-Squares Hierarchy for Max-CUT

Statement:

{‘sentence1’: ‘For every n -vertex graph G , the minimal rank of the geometric quantization over its associated symplectic manifold is equal to the logarithm of the maximum number of distinct solutions to the MAX-CUT problem on G .’, ‘sentence2’: ‘Equivalently, for all n -vertex graphs G , the sum-of-squares hierarchy level required to approximate the MAX-CUT solution is proportional to the minimal rank of the geometric quantization over G .’, ‘sentence3’: ‘This proportionality constant is at least 1 and independent of the graph G .’}

Rationale (proposer’s reasoning):

{‘sentence1’: ‘The conjecture bridges the gap between geometric quantization, which provides a noncommutative description of classical mechanics, and the sum-of-squares hierarchy, a tool used in quantum computing.’, ‘sentence2’: ‘Geometric quantization is expected to reveal structural information about the complexity of solving problems like MAX-CUT, while the sum-of-squares methods are key in understanding the complexity of approximation algorithms.’, ‘sentence3’: ‘This connection could potentially lead to new insights into both fields.’}

Taxonomy category: minimal_rank_geometric_quantization (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 337db5b701cbb60c

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if the ratio between the minimal rank of geometric quantization and the logarithm of the MAX-CUT solutions is within $\pm 10\%$ for all tested graphs, with no seed producing a difference greater than 10%. Falsified if this ratio exceeds $\pm 20\%$ for any graph or any seed produces a difference greater than 10%.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - geometric quantization AND symplectic manifold AND minimal rank - sum-of-squares hierarchy AND Max-CUT problem AND logarithmic approximation - quantization theory AND complexity theory AND geometric quantization over symplectic manifolds

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    # Generate a random graph with n vertices and m edges
    n = 10 + random.randint(0, 20) # Ensure n is at least 5
    m = random.randint(n - 1, n * (n - 1) // 2)
    G = [[0] * n for _ in range(n)]
    edges = set()
    while len(edges) < m:
        u, v = random.sample(range(n), 2)
        if u != v and (u, v) not in edges and (v, u) not in edges:
            G[u][v] = G[v][u] = 1
            edges.add((u, v))

    # Calculate the minimal rank of geometric quantization (simplified as a proxy)
    min_rank = n

    # Compute the MAX-CUT solution (simplified as a proxy)
    max_cut_value = sum(max(sum(row[:i]) for row in G), sum(G[i][j] for j in range(i + 1, n))) / 2

    # Calculate the logarithm of the number of solutions
    log_num_solutions = math.log(max_cut_value + 1)

    # Check if the conjecture holds
    ratio = abs(min_rank - log_num_solutions) / (min_rank + log_num_solutions)
    conjecture_holds = ratio <= 0.1
```

```

    return {
        "metric_name": "Ratio of Minimal Rank to Logarithm of MAX-CUT Solutions",
        "metric_value": ratio,
        "instances_tested": 1,
        "conjecture_holds": conjecture_holds,
        "counterexample": "" if conjecture_holds else f"Graph with {n} vertices and {m} edges"
    }

if __name__ == "__main__":
    seeds = [int(s) for s in sys.argv[1:]] or [2**i - 1 for i in range(5, 30)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_ratio = sum(r["metric_value"] for r in results) / len(results)
    std_ratio = math.sqrt(sum((r["metric_value"] - mean_ratio)**2 for r in results) / len(results))
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_ratio} std={std_ratio} support_fraction={support_fraction}")
    elif any(abs(r["metric_value"]) > 0.2 for r in results):
        first_failing_seed = next(s for s, r in zip(seeds, results) if abs(r["metric_value"]) > 0.2)
        print(f"RESULT: FALSIFIED counterexample='Graph with {n} vertices and {m} edges' first_failing_seed={first_failing_seed}")
    else:
        print(f"RESULT: INCONCLUSIVE support_fraction={support_fraction}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_06b54250.py", line 58, in <module>
    result = run_trial(seed)
              ~~~~~^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_06b54250.py", line 36, in run_trial
    max_cut_value = sum(max(sum(row[:i]) for row in G), sum(G[i][j] for j in range(i + 1, n))) / 2
                      ~~~~~^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_06b54250.py", line 36, in <genexpr>
    max_cut_value = sum(max(sum(row[:i]) for row in G), sum(G[i][j] for j in range(i + 1, n))) / 2
                      ^
NameError: name 'i' is not defined. Did you mean: 'id'?

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code crashed due to an undefined variable 'i', which prevented it from producing any data. As a result, the pre-registered support condition could not be unambiguously met. | next: Review and correct the test code to ensure that all variables are properly defined before running the test again.

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	14802
2	propose	ollama_remote	glm4:latest	0	0	14713
3	preregistration	ollama_remote	glm4:latest	0	0	14494
4	novelty	ollama_remote	glm4:latest	0	0	11117
5	novelty	ollama_remote	glm4:latest	0	0	8820
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	27870
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11140
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9474
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	8216
10	judge	ollama_remote	glm4:latest	0	0	11751

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 132397 ms total latency. Provider mix: {'ollama_remote': 10}

(full prompt+response transcripts available in research/audit/93f863499760.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/93f863499760.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/93f863499760.tar.gz (if generated)